LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 06 – Inheratance



Nama: CALYA ALESITA PUTRI IRAWAN

NIM: 103132400020

Kelas: PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Inheritance merupakan konsep pewarisan dalam Pemrograman Berorientasi Objek (PBO) yang memungkinkan suatu class mewarisi atribut dan method dari class lain. Class yang mewariskan disebut *superclass,* sedangkan class yang menerima pewarisan disebut *subclass.* Inheritance digunakan untuk mengurangi duplikasi kode dan membentuk hubungan antarclass menggunakan keyword extends.

Selain inheritance, terdapat keyword super yang digunakan untuk mengakses atribut, *method*, atau *constructor* dari *superclass*. Modul ini juga membahas *method overloading*, yaitu penggunaan nama method yang sama dengan parameter berbeda, serta *method overriding*, yaitu pendefinisian ulang *method* dari *superclass* pada subclass. Konsep-konsep tersebut membantu membuat program Java lebih terstruktur dan mendukung penerapan polimorfisme.

B. Guided

1. Inheritance

```java
package guided.guided1;

public class A {
    int x;
    int y;

    void tampilXY() {
        System.out.println("Nilai x = "+x+", y = "+y);
    }
}
```

```java
package guided.guided1;

public class B extends A {
    int z;
    int hasil;

    void jumlahXYZ() {
        hasil = x+y+z;

        System.out.println("Jumlah (x+y+z) = " + hasil);
    }

}
```

```java
package guided.guided1;

public class Demo {
    public static void main(String[] args) {
```

```
        A superclass = new A();
        B subclass = new B();

        superclass.x = 3;
        superclass.y = 4;

        System.out.println("superclass: ");
        superclass.tampilXY();

        subclass.x = 1;
        subclass.y = 2;
        subclass.z = 5;

        System.out.println("subclass: ");
        subclass.jumlahXYZ();
    }
}
```

Output :

```
superclass:
Nilai x = 3, y = 4
subclass:
Jumlah (x+y+z) = 8
```

2. Keyword Super

```
[package guided.guided2;

public class Employee {
    private String name;
    String departemen;

    public Employee(String name) {
        this.name = name;
    }

    public void tampilNama() {
        System.out.println("nama: "+ name);
    }
}
```

```
package guided.guided2;

public class Manager extends Employee {
```

```
    private String alamat;

    public Manager(String nama, String alamat) {
        super(nama);
        this.alamat = alamat;
    }

    public void tampilInformasi() {
        super.tampilNama();
        System.out.println("alamat: " + alamat);
        System.out.println("departemen: " + super.departemen);
    }

}
```

```
package guided.guided2;

public class Demo {
    public static void main(String[] args) {
        Manager adi = new Manager("Adi", "Sukabirus");
        adi.departemen = "SDM";
        adi.tampilInformasi();
    }
}
```

Output :

```
nama: Adi
alamat: Sukabirus
departemen: SDM
```

3. Overloading Method

```
package guided.guided3;

public class Lagu {
    String pencipta;
    String judul;

    void IsiParam(String param) {
        judul = param;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1, String param2) {
```

```
        judul = param1;
        pencipta = param2;
    }

    void tampilkan(){
        System.out.println("Judul: "+ judul + ", pencipta: "+
pencipta);
    }
}
```

```
package guided.guided3;

public class Main {
     public static void main(String[] args) {
        Lagu d,e;
        d = new Lagu();
        e = new Lagu();

        d.IsiParam("Lagu 1");
        e.IsiParam("Lagu 2", "aku");

        d.tampilkan();
        e.tampilkan();
    }
}
```

Output :

```
Judul: Lagu 1, pencipta: Tidak dikenal
Judul: Kepastian yang kutunggu, pencipta: GiGi
```

4. Overiding Method

```
package guided.guided4;

public class A {
    public void tampilkan(){
        System.out.println("Method milik class A dipanggil...");
    }
}
```

```
package guided.guided4;

public class B extends A {
     public void tampilkan(){
        super.tampilkan();
        System.out.println("Method milik class B dipanggil...");
    }
```

```
}
```

```
package guided.guided4;

public class Main {
     public static void main(String[] args){
        B overidB = new B();
        overidB.tampilkan();
    }
}
```

Output:

```
Method milik class A dipanggil...
Method milik class B dipanggil...
```

C. Unguided

1. Soal Unguided 1

Buatlah program Java untuk menghitung dan membandingkan estimasi biaya komputasi model machine learning dengan menerapkan inheritance, polymorphism, method overloading, dan method overriding. Program terdiri dari kelas induk model umum, kelas turunan model regresi dan klasifikasi, serta kelas klasifikasi deep learning yang mewarisi kelas klasifikasi.

Setiap kelas memiliki atribut private, constructor, dan method sesuai ketentuan soal. Kelas induk menyediakan method untuk menampilkan informasi dan menghitung biaya komputasi berdasarkan jumlah data latih, baik tanpa parameter maupun dengan parameter epoch. Kelas turunan menambahkan atribut masing-masing dan melakukan overriding atau overloading sesuai aturan.

Pada class Main, buat objek Regresi Linier, Decision Tree, dan Neural Network sesuai data yang ditentukan. Simpan objek dalam satu array bertipe kelas induk, tampilkan informasi dan biaya setiap model menggunakan perulangan, hitung total biaya, lalu tampilkan biaya 10 epoch untuk Neural Network dan Regresi Linier serta biaya 10 epoch dengan batch 50 untuk Regresi Linier.

Jawaban :

```
package unguided;

public class ModelUmum {
    private String namaModel;
    private int jumlahDataLatih;

    public ModelUmum(String namaModel, int jumlahDataLatih) {
```

```
        this.namaModel = namaModel;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    public String getNamaModel() {
        return namaModel;
    }

    public int getJumlahDataLatih() {
        return jumlahDataLatih;
    }

    public void tampilkanInformasi() {
        System.out.println("Model: " + namaModel
                + " | Data latih: " + jumlahDataLatih);
    }

    public int hitungBiaya() {
        return jumlahDataLatih * 2;
    }

    public int hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}
```

```
package unguided;

public class ModelRegresi extends ModelUmum{
    private int jumlahFitur;

    public ModelRegresi(String namaModel, int jumlahDataLatih, int
jumlahFitur) {
        super(namaModel, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    public void tampilkanInformasi() {
        super.tampilkanInformasi();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }
```

```
    public int hitungBiaya(int epoch, int batch) {
        return getJumlahDataLatih() * epoch / batch;
    }
}
```

```
package unguided;

public class ModelKlasifikasi extends ModelUmum{
    private int jumlahKelas;

    public ModelKlasifikasi(String namaModel, int jumlahDataLatih,
int jumlahKelas) {
        super(namaModel, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    public int getJumlahKelas() {
        return jumlahKelas;
    }

    public void tampilkanInformasi() {
        super.tampilkanInformasi();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    public int hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }
}
```

```
package unguided;

public class ModelDeepLearning extends ModelKlasifikasi{
    private int jumlahLayer;

    public ModelDeepLearning(String namaModel, int jumlahDataLatih,
int jumlahKelas, int jumlahLayer) {
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    public void tampilkanInformasi() {
        super.tampilkanInformasi();
        System.out.println("Jumlah layer: " + jumlahLayer);
    }

    public int hitungBiaya() {
```

```
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}
```

```
package unguided;

public class Main {
    public static void main(String[] args) {
        ModelUmum[] daftarModel = {
            new ModelRegresi("Regresi Linier", 500, 4),
            new ModelKlasifikasi("Decision Tree", 800, 3),
            new ModelDeepLearning("Neural Network", 1000, 5, 3)
        };

        int totalBiaya = 0;

        for (ModelUmum model : daftarModel) {
            model.tampilkanInformasi();

            int biaya = model.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");

            totalBiaya += biaya;
        }

        System.out.println("Total biaya: " + totalBiaya);

        ModelUmum neuralNetwork = daftarModel[2];
        ModelUmum regresiLinier = daftarModel[0];

        System.out.println("Biaya 10 epoch (Neural Network): "
                + neuralNetwork.hitungBiaya(10));

        System.out.println("Biaya 10 epoch (Regresi Linier): "
                + regresiLinier.hitungBiaya(10));

        ModelRegresi regresi = (ModelRegresi) daftarModel[0];

        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier):
"
                + regresi.hitungBiaya(10, 50));
    }
}
```

Output :

```
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
```

Penjelasan :

Pada program ini adalah menghitung estimasi biaya komputasi dari tiga model *machine learning*, yaitu *Regresi Linier, Decision Tree*, dan *Neural Network*. Program terdiri dari class ModelUmum sebagai kelas induk, ModelRegresi dan ModelKlasifikasi sebagai kelas turunan, serta ModelDeepLearning yang merupakan turunan dari ModelKlasifikasi. Setiap kelas memiliki atribut dan method sesuai ketentuan soal untuk menampilkan informasi model dan menghitung biaya komputasi berdasarkan jumlah data latih, jumlah fitur, jumlah kelas, jumlah epoch, ukuran batch, dan jumlah layer.

Pada class Main, ketiga objek model disimpan dalam satu *array* bertipe ModelUmum dan diproses menggunakan perulangan untuk menampilkan informasi serta menghitung total biaya komputasi. Hasil program menunjukkan biaya *Regresi Linier* sebesar 1.040, *Decision Tree* sebesar 4.800, dan *Neural Network* sebesar 10.600, sehingga total biaya komputasi adalah 16.440. Program juga berhasil menampilkan biaya 10 epoch untuk *Neural Network* sebesar 10.000, biaya 10 epoch untuk *Regresi Linier* sebesar 5.000, serta biaya 10 epoch dengan batch 50 untuk *Regresi Linier* sebesar 100.

D. Kesimpulan

Berdasarkan praktikum Modul 6 tentang Inheritance, dapat disimpulkan bahwa konsep pewarisan memungkinkan suatu *class* mewarisi atribut dan method dari *class* lain sehingga program menjadi lebih terstruktur dan mengurangi pengulangan kode. Melalui Guided dan Unguided, dipelajari penggunaan extends dan super, serta penerapan method overloading, overriding, dan polymorphism dalam Java. Pada Unguided, konsep tersebut diterapkan untuk menghitung estimasi biaya komputasi beberapa model *machine learning* menggunakan kelas induk, kelas turunan, dan array. Hasil program menunjukkan bahwa

perhitungan biaya setiap model dan total biaya komputasi berhasil dilakukan sesuai dengan ketentuan soal.